FITBUDDY

PERSONAL FITNESS AND HEALTH MANAGEMENT SYSTEM

PROJECT DOCUMENTATION

Submitted by

NETHRA P

NM ID: EB83E5FE249BCB29A484A2F2FAD306E4

HASINI M

NM ID: 6EDA988EA2776A9B2734FE34FDE2A05D

HEMALATHA S

NM ID: A05462DF5176AEE927241662DA6291EA

MUTHULAKSHMI S

NM ID: C02CB1CA6D51F93AC604DC60EC3B06E1

ACKNOWLEDGEMENT

We express our sincere gratitude to our faculty members, department and institution for their valuable guidance and support throughout the development of FitBuddy. We also thank our friends and family for their encouragement and support during the planning, implementation and testing of the project.

ABSTRACT

FitBuddy is a web-based personal fitness and health management system designed to provide users with a simple platform for accessing and managing fitness-related information and activities. The project uses a Python-based FastAPI backend and a browser-based interface. The application is designed around a modular structure so that fitness-related features, user interactions and data management can be maintained in an organized manner. The system is intended to provide an accessible interface and a foundation for future personalized recommendations, progress visualization, reminders and cloud deployment.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Existing System
5. Proposed System
6. Scope of the Project
7. System Requirements
8. Technologies Used
9. System Architecture
10. System Modules
11. Database Design
12. Implementation
13. User Interface and Application Outputs
14. API and Backend
15. Testing
16. Results
17. Advantages
18. Limitations
19. Future Enhancements
20. Conclusion
21. References

1. INTRODUCTION

Fitness management involves maintaining regular physical activity, monitoring progress and organizing relevant personal fitness information. Many users rely on separate applications, manual notes or disconnected services. FitBuddy is developed as a centralized web application that brings fitness-related functions into one platform.

2. PROBLEM STATEMENT

Users may find it difficult to maintain consistent and organized fitness information when activities and records are managed manually or across multiple applications. The project addresses this need by providing a web-based system with a single interface for fitness-related management and future personalization.

3. OBJECTIVES

- To develop a simple and user-friendly fitness management web application.
- To provide a centralized platform for fitness-related activities and information.
- To implement a Python-based backend for handling application requests.
- To organize application data using a structured database approach.
- To provide a foundation for future personalized fitness features.
- To test the application locally through a web browser.

4. EXISTING SYSTEM

In a conventional approach, users may use multiple applications or manual records to manage fitness information. This can make information difficult to organize and may not provide a single interface for the user's fitness-related activities.

5. PROPOSED SYSTEM

FitBuddy proposes a centralized web application in which the user interacts with a browser-based interface while the FastAPI backend handles requests and application logic. A database can be used to persist relevant application data. The modular design makes the system easier to extend.

6. SCOPE OF THE PROJECT

- Web-based access to FitBuddy functionality.
- Fitness-related user interaction and information management.
- Backend API processing using FastAPI and Uvicorn.
- Database-backed application functionality.
- Future support for dashboards, progress tracking and personalization.

7. SYSTEM REQUIREMENTS

7.1 Hardware Requirements

• Laptop or desktop computer

• Minimum 4 GB RAM recommended

• Sufficient storage for source code and database

• Internet connection when online services are used

7.2 Software Requirements

• Windows operating system

• Python

• FastAPI

• Uvicorn

• HTML/CSS/JavaScript as used by the interface

• Visual Studio Code

• SQLite/database component if configured in the project

8. TECHNOLOGIES USED

Technology	Purpose
Python	Backend programming and application logic
FastAPI	Web API/backend framework
Uvicorn	ASGI server used to run the application
HTML/CSS/JavaScript	Browser-based user interface
SQLite / database	Structured storage where configured
Visual Studio Code	Development environment

9. SYSTEM ARCHITECTURE

The general architecture of FitBuddy follows a client-server model. The user accesses the application through a web browser. Requests are handled by the FastAPI application running through Uvicorn. Application logic communicates with the database where persistent data is required.

User → Web Browser → FastAPI Backend → Application Logic → Database

Database Response → FastAPI Backend → Web Browser → User

10. SYSTEM MODULES

User Interface Module: Provides the pages and controls through which users interact with FitBuddy.

Backend/API Module: Handles browser requests, routing and application logic using FastAPI.

Fitness Management Module: Provides the project's fitness-related functionality and information.

Database Module: Stores and retrieves structured application data where database persistence is configured.

Server Module: Runs the FastAPI application using Uvicorn for local development and testing.

11. DATABASE DESIGN

The project includes a database component. The database file observed during development is named fitbuddy.db. The exact table structure should be maintained consistently with the final source code.

12. IMPLEMENTATION

The application is implemented as a Python web application. During local execution, the FastAPI application is started with Uvicorn. The development server used for testing is accessible through the local address http://127.0.0.1:8000. The application startup was verified when the terminal displayed “Application startup complete.”

13. USER INTERFACE AND APPLICATION OUTPUTS

The three supplied FitBuddy output PDFs are included below exactly once each. Each PDF is represented by its original pages, in order. No duplicate copy of any output is included.

Output PDF 1

Output PDF 1 – Page 1

💪 FitBuddy
Home Feedback All Users

💪 FitBuddy - AI Workout Generator

Generate your personalized AI-powered fitness plan

Name

sandhiya

User ID

10

Age

21

Weight (kg)

40

Fitness Goal

weight loss and muscle gain

Workout Intensity

High

Generate Plan

Output PDF 2

Output PDF 2 – Page 1

💪 FitBuddy
Home Feedback All Users

Personalized Workout Plan

Name: sandhiya

User ID: 10

Age: 21

Weight: 40.0 kg

Goal: weight loss and muscle gain

Intensity: High

Description

Once the user submits their details, the application navigates to a personalized workout plan page. This section displays a summary of the user's input including their Name, User ID, Age, Weight, Fitness Goal, and Selected Intensity.

The personalized plan is generated based on the user's fitness requirements and selected workout intensity.

🏋️ Workout Plan

```
7-Day Personalized Workout Plan

User: sandhiya
Age: 21
Weight: 40.0 kg
Goal: weight loss and muscle gain
Intensity: High

Day 1 - Full Body Workout
• Warm-up: 5-10 minutes
• Squats: 3 sets × 12 reps
• Push-ups: 3 sets × 10 reps
• Lunges: 3 sets × 10 reps
• Plank: 3 sets × 30 seconds
• Cool-down: 5 minutes

Day 2 - Lower Body
• Warm-up: 5-10 minutes
• Bodyweight Squats: 3 × 15
• Lunges: 3 × 12
• Glute Bridges: 3 × 15
• Calf Raises: 3 × 15
• Stretching: 5 minutes

Day 3 - Cardio & Core
• Jumping Jacks: 3 × 30 seconds
• High Knees: 3 × 30 seconds
• Mountain Climbers: 3 × 20
• Bicycle Crunches: 3 × 15
```

Output PDF 3

Output PDF 3 – Page 1

💪 FitBuddy
Home Feedback All Users

Feedback

Help us improve your personalized fitness plan.

✅ Your plan has been updated based on your feedback!

User ID

10

Your Feedback

The workout plan is very useful and easy to follow. I would like more beginner-friendly exercises and a few additional rest day options. Overall, the personalized plan is helpful and well organized.

Submit Feedback

14. API AND BACKEND

FastAPI is used to define and process application routes. The supplied output documents show the local application routes used during testing, including the home page, workout generation page and feedback page.

Method	Endpoint	Purpose
GET	/	Application home route
GET/POST	/generate	Generate/display personalized workout plan
GET/POST	/feedback	Collect and process user feedback

15. TESTING

Testing included server startup, opening the home page, generating the personalized workout plan and submitting feedback. The supplied output documents provide visible evidence of these completed flows.

16. RESULTS

The FitBuddy application was successfully run in the local development environment. The supplied outputs show the AI workout generator, a personalized 7-day workout plan, nutrition and safety information, a feedback section, and a confirmation message indicating that the plan was updated based on feedback. The workout output records the entered profile as Name: sandhiya, User ID: 10, Age: 21, Weight: 40.0 kg, Goal: weight loss and muscle gain, and Intensity: High.

17. ADVANTAGES

• Simple browser-based access.

• Centralized fitness-related functionality.

• Modular backend structure.

• FastAPI provides a lightweight and organized API layer.

• Can be extended with additional fitness and personalization features.

18. LIMITATIONS

• The current development setup is locally hosted unless the project is deployed to a cloud server.

• Some advanced personalization and analytics features may require additional implementation.

• The exact capabilities depend on the features implemented in the final source code.

19. FUTURE ENHANCEMENTS

• Deploy FitBuddy to a cloud platform and provide a public HTTPS URL.

• Add personalized workout and fitness recommendations.

• Add progress charts and visual analytics.

• Integrate wearable devices or activity data where appropriate.

• Add notifications and fitness reminders.

• Develop a mobile application.

• Improve authentication, authorization and data security.

20. CONCLUSION

FitBuddy demonstrates the development of a web-based fitness and health management application using a Python backend and browser-based interface. The project provides a structured foundation for managing